A MAJOR PROJECT REPORT

On

VEHICLE PARKING DETECTION USING MASK RCNN

Submitted In Partial Fulfillment of the Requirements for the Award of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

(INTERNET OF THINGS)

By

E. PURNA CHANDAR (21C31A6920)

UNDER THE GUIDANCE OF

Mr. SD Sameer

Assistant professor – Dept of CSE(IOT)

Department of Computer Science and Engineering

(INTERNET OF THINGS)

BALAJI INSTITUTE OF TECHNOLOGY & SCIENCE

Laknepally, Narsampet, Warangal (Rural)-506331, Telangana State, India (Autonomous)

Accredited by NBA (UG-CE, EEE, ECE, ME & CSE Programmes) & NAAC A+ Grade (Affiliated to JNTU Hyderabad and Approved by the AICTE, NewDelhi)

2024-2025

BALAJI INSTITUTE OF TECHNOLOGY & SCIENCE

Laknepally, Narsampet, Warangal (Rural)-506331, Telangana State, India (Autonomous)

Accredited by NBA (UG-CE, EEE, ECE, ME & CSE Programmes) & NAAC A+ Grade (Affiliated to JNTU Hyderabad and Approved by the AICTE, New Delhi)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(INTERNET OF THINGS)

CERTIFICATE

This is to certify that E. Purna chandar (21C31A6920), of B. Tech (CSO IV/II) has satisfactorily completed the Major project work entitled "Vehicle parking detection using mask RCNN" in the partial fulfilment of the requirements of the B. Tech degree during this academic year 2024-2025.

Signature of the Guide
Mr. SD Sameer
Assistant Professor ,
Department of IoT,
BITS, Narsampet-506331

Signature of the Head of the Department
Dr. M. SRIDHAR
Associate Professor & Head,
Department of IoT,
BITS,Narsampet-506331

External Examiner

ACKNOWLEDGEMENT

I thank the almighty for giving me the courage and perseverance in completing the project . This project itself is an acknowledgement for all those people who have given me their heart full co-operation in making my project a success.

I register a deep sense of gratitude to **Dr. V. S. Hariharan, Principal** for making me available the necessary infrastructure.

I am greatly indebted to **Dr. M. Sridhar, Associate. Professor - Head of the Department** for his valuable advices at every stage of this work. I feel that without his supervision and many hours of devoted guidance, stimulating and constructive criticism, and this thesis would never have come out in this form.

I am also thankful to **Mr. Sd.S.Sameer, Asst. Prof, and Project Coordinator** for providing the excellent facilities, motivation and valuable guidance throughout the project work. With his co-operation and encouragement, I completed the project work in time.

I take this opportunity to express my deep and sincere gratitude to the project guide **Mr. Sd.S.Sameer, Asst. Prof** Balaji Institute of Technology & Science.

Last but not least I would like to express my deep sense of gratitude and earnest thanks giving to my dear parents for their moral support and heartfelt co-operation in doing the project. I would also like to thank all the teaching and non-teaching staff and my friends, whose who direct or indirect help has enabled us to complete this work successfully.

E.PURNA CHANDAR (21C31A6920)

ABSTRACT

This project presents a multicamera system for vehicles detection and their corresponding mapping into the parking spots of a parking lot. Parking lots are a widely used service where a great investment is made every year. The management of these car parks is very expensive and in many cases complex, especially in the case of those that have many places such as airports or large commercial areas. Solving this problem using computer vision promises a number of advantages over intrusive sensors like induction loops or other weight-in-motion sensors. The proposed system has been designed for realistic scenarios considering different cases of occlusion, illumination changes, and different climatic conditions, it has been targeted with the condition that existing parking security cameras can be used, avoiding the deployment of new cameras or other sensors infrastructures. For design and validation, a new multicamera data set has been recorded.

The system is based on existing object detectors (the results of two of them are shown) and different proposed post processing stages. The results clearly show that the proposed system works correctly in challenging scenarios including almost total occlusions, illumination changes, and different weather conditions.

TABLE OF CONTENTS

1	**Introduction…….....**	**1**
	1.1Software requirements.....	1
	1.2Hardware requirements.....	2
2	**Feasibility study……**	**3**
	2.1 Economic feasibility.....	3
	2.2 Technical feasibility.....	4
	2.3 Social feasibility.....	4
3	**Literature survey…….**	**5**
4	**System analysis……**	**9**
	4.1 Existing system.....	9
	4.1.1 Disadvantages of existing system.....	9
	4.2 Proposed system.....	9
	4.2.1 Advantages of proposed system.....	10
	4.3 Functional requirements.....	10
	4.4 Non-Functional requirements.....	10
5	**System design……..**	**12**
	5.1 System architecture.....	12
	5.2 UML diagrams.....	14
6	**Implementation…….**	**20**
	6.1 Modules.....	20
	6.2 Sample code.....	21
7	**Software environment……**	**30**
8	**System testing……**	**40**
	8.1 Testing strategies.....	40
	8.2 Test cases.....	41

9	**Screenshorts……..**	**42**
10	**Conclusion……**	**45**
11	**References…….**	**47**

LIST OF FIGURE

5.1.1	System architecture.....	12
5.1.2	Dataflow diagram.....	13
5.2.1	Usecase diagram.....	15
5.2.2	Class diagram.....	16
5.2.3	Activity diagram.....	17
5.2.4	Sequence diagram.....	18
5.2.5	Component diagram.....	19
9.1	Command prompt.....	42
9.2	Sign up page.....	42
9.3	Sign in page.....	43
9.4	Home page after sign in.....	43
9.5	Output Detection.....	44
9.6	Anaconda prompt.....	44

1. INTRODUCTION

Parking lots are a widely used service where a great investment is made every year. The management of these car parks is very expensive and in many cases complex, especially in the case of those that have many places such as airports or large commercial areas. Solving this problem using computer vision promises a number of advantages over intrusive sensors like induction loops or other weight-in-motion sensors [1]. In addition, a vision-based system may provide many value-added services, like parking space guidance and video surveillance [2]. Such systems allow the decongestion of crowded parking areas, directing vehicles to areas with lower occupancy, guiding the vehicles by a faster route. Surveillance cameras are readily available in most car parking lots, so in many cases the solution is only to adequately process the information available from the already existing cameras, or complete the deployment by adding some cameras to have a full coverage that allows the system to operate. The previously developed systems are mainly based on image segmentation or machine learning (SVMs, NN) over spot patches, but due to the evolution in the last years of object detection algorithms, it is possible to use the detections of these algorithms for the proper operation of automatic parking management systems.

1.1 SOFTWARE REQUIREMENTS

The functional requirements or the overall description documents include the product perspective and features, operating system and operating environment, graphics requirements, design constraints and user documentation.

The appropriation of requirements and implementation constraints gives the general overview of the project in regards to what the areas of strength and deficit are and how to tackle them.

- Python idel 3.7 version (or)
- Anaconda 3.7 (or)
- Jupiter (or)
- Google colab

1.2 HARDWARE REQUIREMENTS

Minimum hardware requirements are very dependent on the particular software being developed by a given Enthought Python / Canopy / VS Code user. Applications that need to store large arrays/objects in memory will require more RAM, whereas applications that need to perform numerous calculations or tasks more quickly will require a faster processor.

- Operating system : windows, linux
- Processor : minimum intel i3
- Ram : minimum 4 gb
- Hard disk : minimum 250gb

2. FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

2.1 ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

2.2 TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

2.3 SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

3. LITERATURE SURVEY

3.1 An algorithm for parking lot occupation detection,:

https://sci-hub.hkvisa.net/10.1109/cisim.2008.53

ABSTRACT: This paper presents unsupervised vision-based system for parking lot occupancy detection. The proposed method exhibit low computation complexity and use just a few frames per minute. Method is based on three main processing stages. In the first section, raw image acquired by camera system is preprocessed. Shadows in the image are significantly attenuated or completely removed. The image distortion is corrected subsequently. In the following step optimal correspondences between one or more stationary cameras and visible parking places are established. During this process occlusions are taken in account. Finally, a parking place status is evaluated. Acquired information about parking lot occupancy can be served to another system, e.g. intelligent transportation system. Experimental results from both artificial and real scenes show promising results even on quite challenging conditions. Nonetheless a scope for improvement of presented method is given.

3.2 A hierarchical Bayesian generation framework for vacant parking space detection

https://sci-hub.hkvisa.net/10.1109/tcsvt.2010.2087510

ABSTRACT: In this paper, from the viewpoint of scene understanding, a three-layer Bayesian hierarchical framework (BHF) is proposed for robust vacant parking space detection. In practice, the challenges of vacant parking space inference come from dramatic luminance variations, shadow effect, perspective distortion, and the inter-occlusion among vehicles. By using a

hidden labeling layer between an observation layer and a scene layer, the BHF provides a systematic generative structure to model these variations. In the proposed BHF, the problem of luminance variations is treated as a color classification problem and is tackled via a classification process from the observation layer to the labeling layer, while the occlusion pattern, perspective distortion, and shadow effect are well modeled by the relationships between the scene layer and the labeling layer. With the BHF scheme, the detection of vacant parking spaces and the labeling of scene status are regarded as a unified Bayesian optimization problem subject to a shadow generation model, an occlusion generation model, and an object classification model. The system accuracy was evaluated by using outdoor parking lot videos captured from morning to evening. Experimental results showed that the proposed framework can systematically determine the vacant space number, efficiently label ground and car regions, precisely locate the shadowed regions, and effectively tackle the problem of luminance variations.

3.3 A vehicle parking detection method using image segmentation

https://www.researchgate.net/publication/230500234_A_vehicle_parking_detection_method_using_image_segmentation

ABSTRACT: A method of individual vehicle detection using grayscale images acquired from a high position is proposed for guidance of incoming vehicles to vacant cells in a parking lot and other similar purposes. With the proposed method, each image region corresponding to a cell is fragmented according to density (gray level), and the distribution of segment area is analyzed to decide if a vehicle is present. Reference images taken in vacant state are not needed, hence the method can be easily applied to parking lots in continuous service. Shape features are not employed, hence detection is performed independent of car shape. The proposed method was tested on an

actual outdoor parking lot during 4 days with different weather conditions from sunrise through sunset. The results confirmed the efficiency of the proposed method, with the detection rate being over 98.7%.

3.4 An automatic monitoring approach for unsupervised parking lots in outdoors

https://www.researchgate.net/publication/4222551_An_automatic_monitoring_approach_for_unsupervised_parking_lots_in_outdoors

ABSTRACT: A video-based monitoring approach for outdoor parking lots has been developed in this paper. Since parking lots located in outdoor were the open and widespread spaces, moving objects in grabbed images were too small to obtain the detail and recognized image for the object, identification, object behavior analysis, or illegal event alarming. A dual-camera device was designed and calibrated manually. Using the calibrated parameters, multiple target images with high quality can be grabbed from widespread open spaces. The tracking process for a specified target can be easily switched to another when multiple objects appear in the monitoring space. All of them are stored in video-based databases of DVR systems. In addition, the object images are processed for the furthermore retrieval. Some experimental results were given to show the validity of our proposed approaches.

3.5 A vision-based parking lot management system

https://www.researchgate.net/publication/4262674_A_Vision-Based_Parking_Lot_Management_System

ABSTRACT: Goals of parking lot management system include counting the number of parked vehicles, monitoring the changes of the parked vehicles over the time, and identifying the stalls available. To decrease the cost of the production, an integrated vision-based system is a good choice. In this paper, we propose a vision-based parking management system to manage an outdoor

parking lot by four cameras set up at loft of buildings around it, sending information, including real-time display, to database of ITS center via internet. This system enables drivers to find parking spaces available or monitoring the parking lot where they parked their cars easily by wireless communication device. To increase accuracy, in the beginning, color manage is done to all input images, maintaining color consistency. Then, an adaptive parking lot background model is generated. The adequate color of each parking space is found out using statistical method in color image sequences captured by a camera, and foreground is extracted based on color information. The result will be further modified by shadow detection based on luminance analysis. Vision-based parking management system can manage large area by just several cameras. Adjusting position of the camera can easily make this system suitable for most cases. Besides, this system is endurable and is easy-installed because of its simple equipment.

4. SYSTEM ANALYSIS

4.1 EXISTING SYSTEM:

The majority of parking lots have surveillance cameras readily available, so in many cases, the solution consists only of processing the information obtained from the already installed cameras appropriately or finishing the deployment by adding additional cameras to have a full coverage that permits the system to function. Since object detection algorithms have advanced over the past few years, it is now possible to use the detections of these algorithms for the proper operation of automatic parking management systems. The previously developed systems are primarily based on image segmentation or machine learning (SVMs, NN) over spot patches.

4.1.1 DISADVANTAGES OF EXISTING SYSTEM:

1. Approaches from the state-of-theart system, which work properly in controlled scenarios, have been validated using small amount of sequences and without more challenging realistic conditions (illumination changes and different weather conditions).
2. On the other hand, most of them are not complete systems, but provide only parts of them, usually detectors.

4.2 Proposed System:

The International Pittsburgh Airport parking lot has been chosen as the target scenario for the proposed system under the condition that existing parking security cameras can be used, avoiding the deployment of new cameras or other sensor infrastructures. The proposed system has been designed for

realistic scenarios taking different cases of occlusion, illumination changes, and different climatic conditions into consideration. A new multicamera data set has been recorded for design and validation. The system is based on various proposed postprocessing stages and current object detectors (the results of two of them are displayed).

4.2.1 Advantages of proposed system:

1. The results clearly show that the proposed system works correctly in challenging scenarios including almost total occlusions, illumination changes, and different weather conditions.
2. The main advantage of the proposed system over the image segmentation based systems is the robustness against variable background.
3. Another advantage of detection based systems is the capacity to withstand "object occlusions". Although some of the existing systems

4.3 FUNCTIONAL REQUIREMENTS

1.Data Collection

2.Data Preprocessing

3.Training And Testing

4.Modiling

5.Predicting

4.4 NON FUNCTIONAL REQUIREMENTS

NON-FUNCTIONAL REQUIREMENT (NFR) specifies the quality attribute of a software system. They judge the software system based on Responsiveness, Usability, Security, Portability and other non-functional standards that are critical to the success of the software system. Example of nonfunctional requirement, *"how fast does the website load?"* Failing to meet

non-functional requirements can result in systems that fail to satisfy user needs. Non- functional Requirements allows you to impose constraints or restrictions on the design of the system across the various agile backlogs. Example, the site should load in 3 seconds when the number of simultaneous users are > 10000. Description of non-functional requirements is just as critical as a functional requirement.

- Usability requirement
- Serviceability requirement
- Manageability requirement
- Recoverability requirement
- Security requirement
- Data Integrity requirement
- Capacity requirement
- Availability requirement
- Scalability requirement
- Interoperability requirement
- Reliability requirement
- Maintainability requirement
- Regulatory requirement
- Environmental requirement

5. SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE:

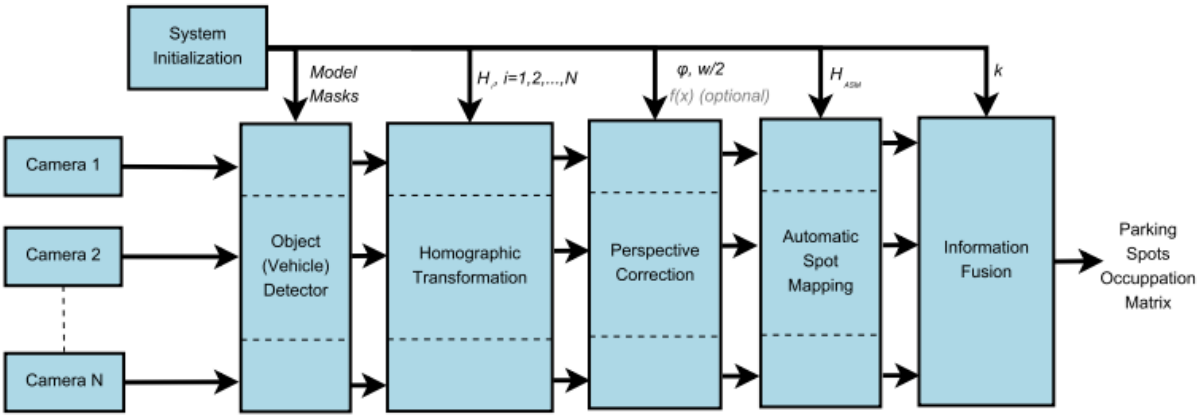

Fig.5.1.1 System architecture

DATA FLOW DIAGRAM:

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process.
3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.

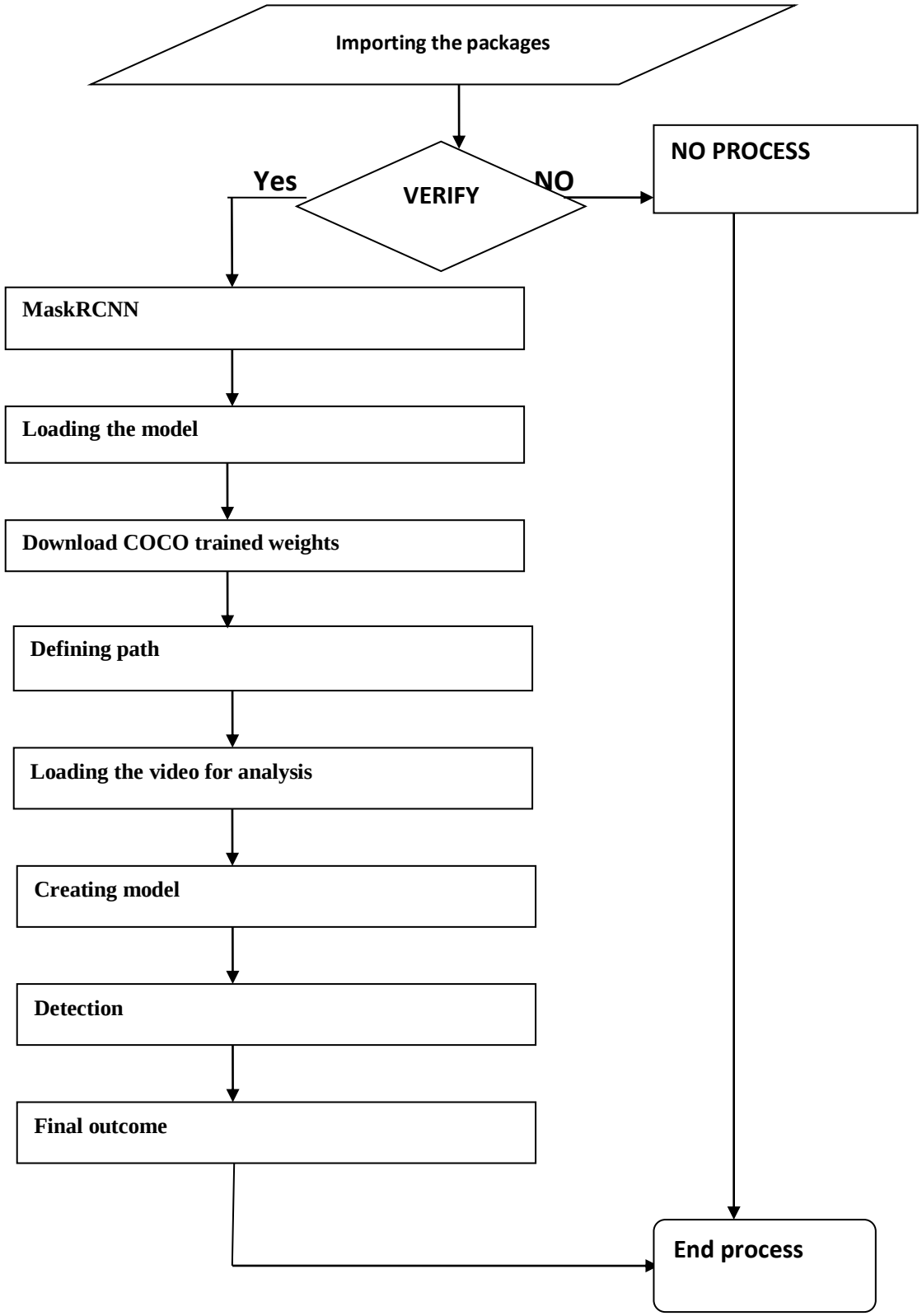

Fig.5.1.2 Dataflow diagram

5.2 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

Use case diagram:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

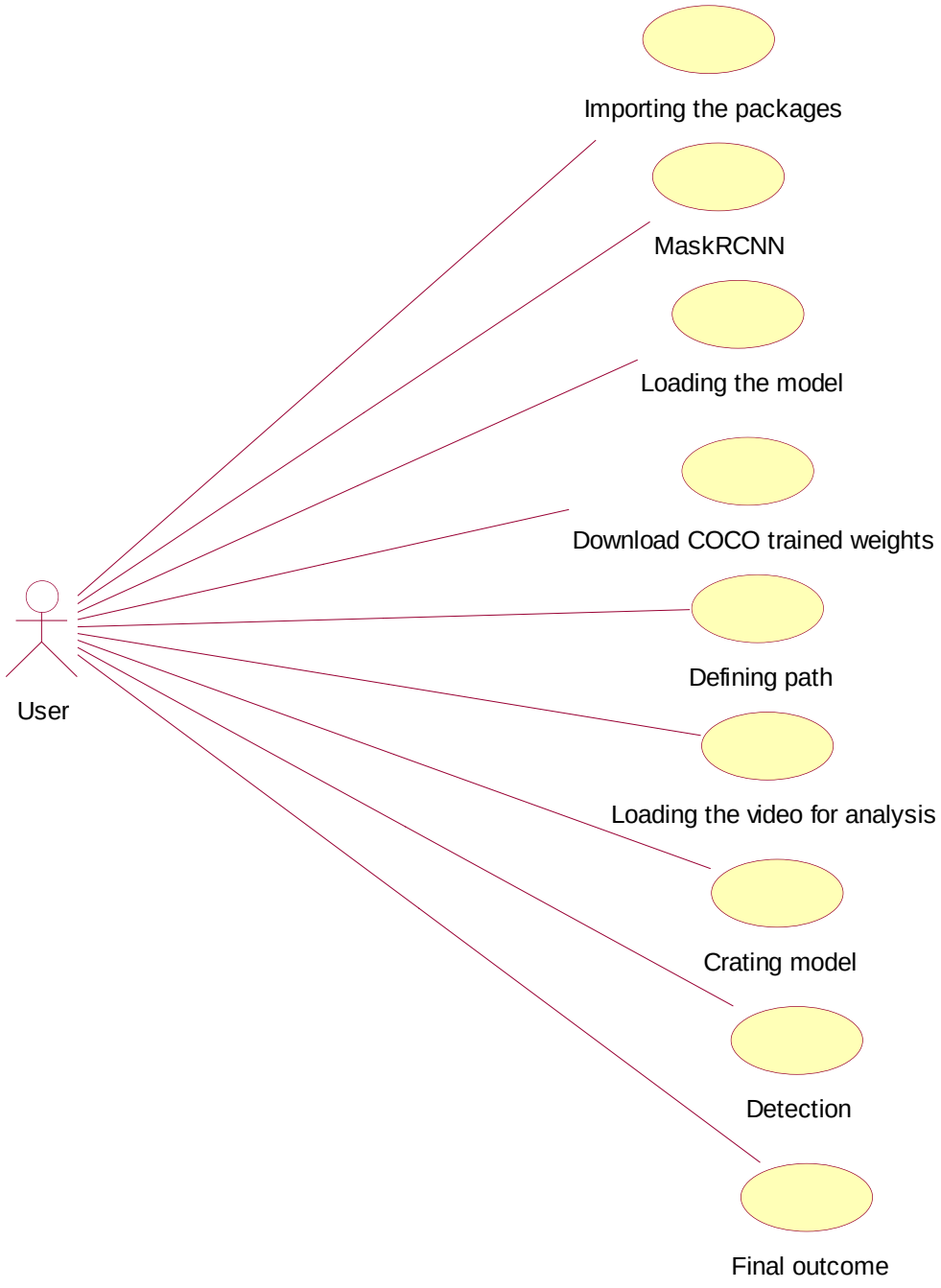

Fig.5.2.1 Usecase diagram

Class diagram:

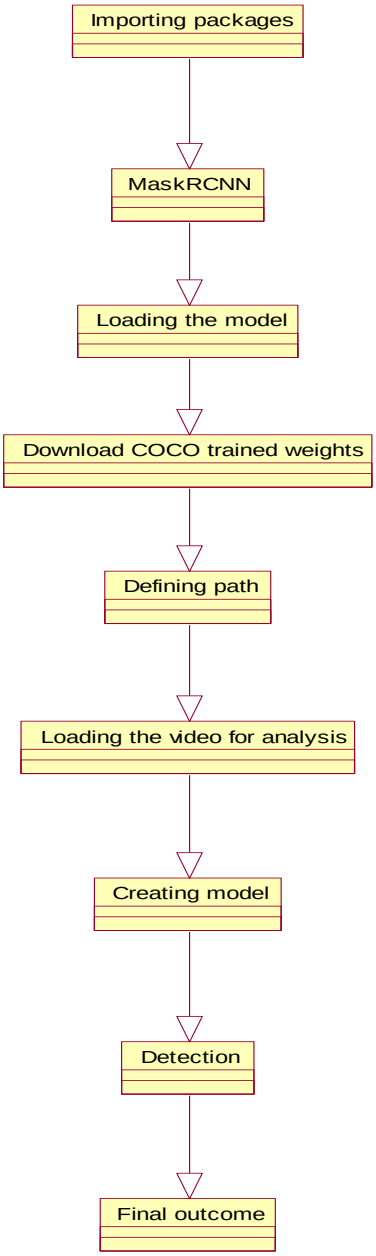

Fig.5.2.2 Class diagram

Activity diagram:

The process flows in the system are captured in the activity diagram. Similar to a state diagram, an activity diagram also consists of activities, actions, transitions, initial and final states, and guard conditions.

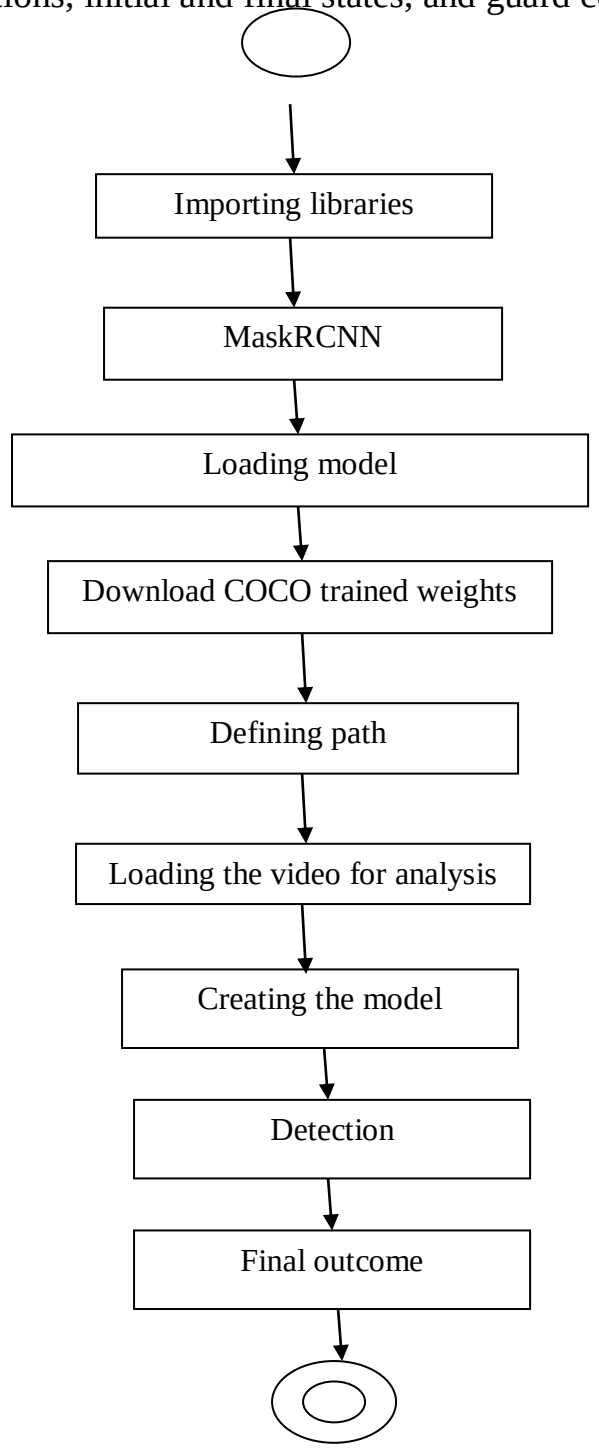

Fig.5.2.3 Activity diagram

Sequence diagram:

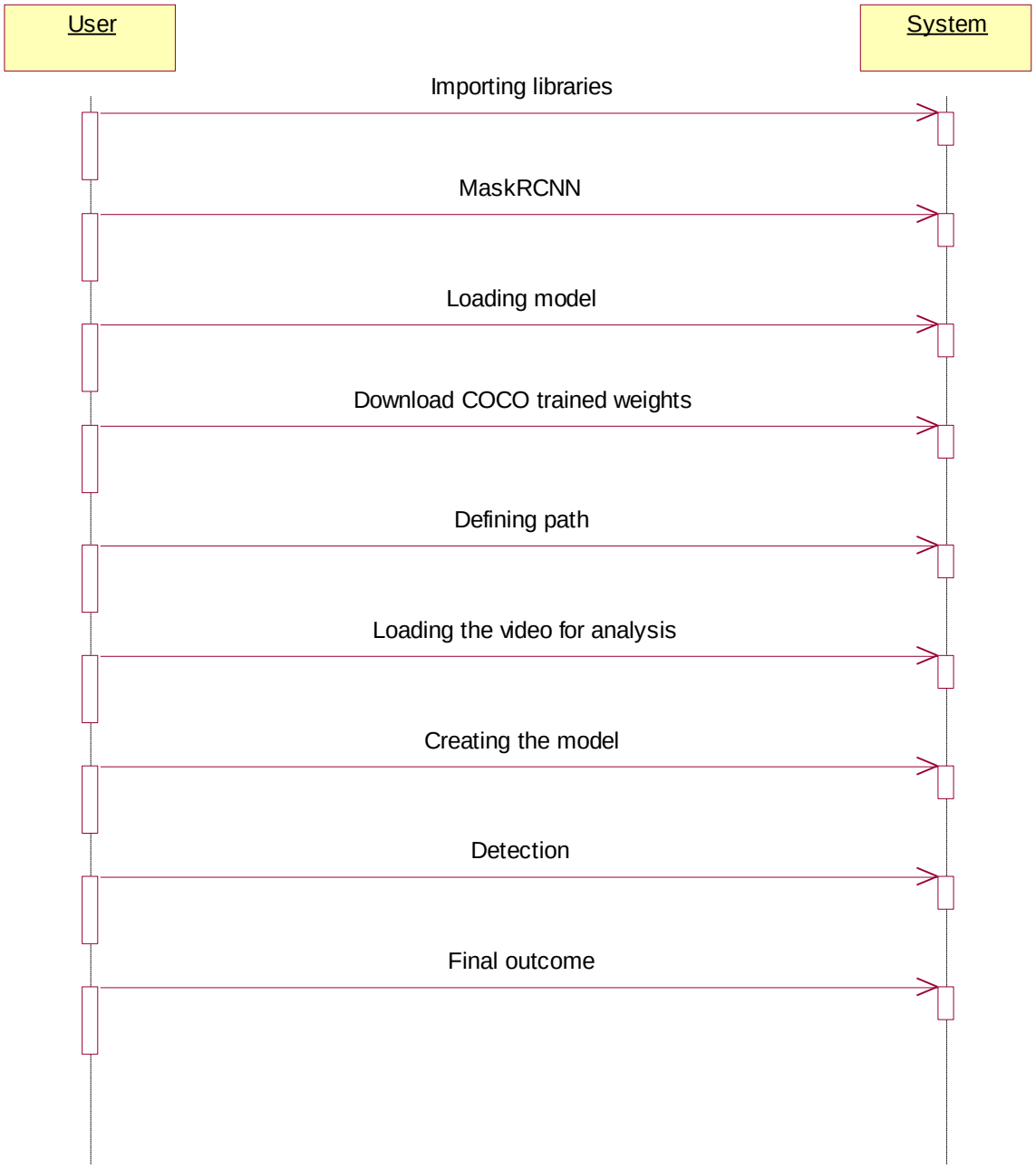

Fig.5.2.4 Sequence diagram

Component diagram:

The component diagram represents the high-level parts that make up the system. This diagram depicts, at a high level, what components form part of the system and how they are interrelated. A component diagram depicts the components culled after the system has undergone the development or construction phase.

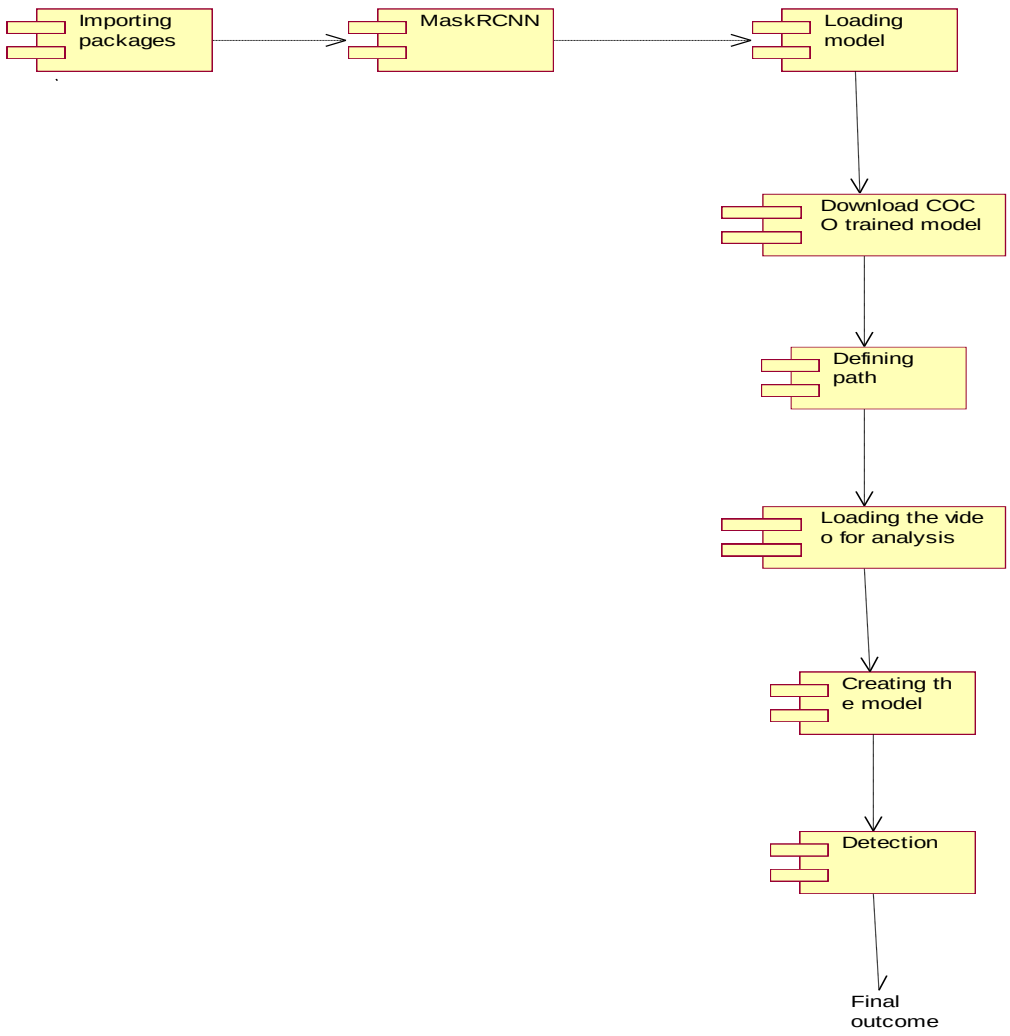

Fig.5.2.5 Component diagram

6. IMPLEMENTATION

6.1 MODULES:

1.importing the packages: Using this will import necessary packages from our program

2.Defining the class fuction of MaskRCNN - Configuration that will be used by the Mask-RCNN library

3. Function for Boxes - Filter a list of Mask R-CNN detection results to get only the detected cars / trucks

- If the detected object isn't a car / truck, skip it

4. Loading the Model : using this model will be loaded

5. Download COCO trained weights from Releases: Downloading weights

6. Defining the path where image to run detection and analysis

7. Loading the video for analysis: Video will be loaded

8. creating the MaskRcnn model and loading the pre trained model

9. Load the video file we want to run detection and Loop over each frame of video

- Convert the image from BGR color (which OpenCV uses) to RGB color

- Run the image through the Mask R-CNN model to get results.

- Mask R-CNN assumes we are running detection on multiple images.

- The r variable will now have the results of detection

- Filter the results to only grab the car / truck bounding boxes

- Draw each box on the frame

- Show the frame of video on the screen

-Hit 'q' to quit

10. Final outcome is displayed in frontend with help of Flask Framework

ALGORITHMS:

MASKRCNN- Mask R-CNN is a state of the art model for instance segmentation, developed on top of Faster R-CNN. Faster R-CNN is a region-based convolutional neural networks, that returns bounding boxes for each object and its class label with a confidence score. The pre-processed and annotated images are used to train and validate the Mask R-CNN Classifier. Our experimental results show that damage can be classified efficiently with 95.13% accuracy on a custom dataset and 96.87% on randomly picked images. Mask R-CNN uses a fully connected network to predict the mask. This ConvNet takes an RoI as input and outputs the m*m mask representation. We also upscale this mask for inference on input image and reduce the channels to 256 using 1*1 convolution.

6.2 SAMPLE CODE:

```
impor
t
argpa
rse
          import sys
          from heapq import heapify, heappush, heappop
          from Models.Car import Car
          from Models.ParkingTicket import ParkingTicket
          class ParkingManagement:
              """
              This is the Main Class for Managing Automated Parking
```

```
Ticketing System.
    """
    def __init__(self):
        """
        This constructor method is used to initialize the
capacity, available_parking_slots and
occupied_parking_slots
        parameters of the class.
        capacity -> It is initialized to zero.
        available_parking_slots -> It is initialized to
an empty list.
        occupied_parking_slots -> It is initialized to an
empty dictionary.
        """
        self.capacity = 0
        self.available_parking_slots = []
        self.occupied_parking_slots = {}
    def create_parking_slots(self, max_capacity):
        """
        This method takes in the max capacity of the
parking lot and returns a min heap for all available
parking lots.
        :param max_capacity:int It denotes the maximum
capacity of the parking lot.
        :return: status:boolean It returns True if a min
heap containing parking lots in the range [1..capacity],
        is successfully created otherwise it return False
if an exception is thrown in the process.
        """
        try:
            self.capacity = max_capacity
            # Creating empty available_parking_lots heap
            heapify(self.available_parking_slots)
            # Adding parking_lots_index in the range
[1..capacity] to the available_parking_lots_heap using
heappush
            # function.
            for parking_slots_index in range(1,
self.capacity + 1):
                heappush(self.available_parking_slots,
parking_slots_index)
            return True
        except Exception as exception:
            print(exception)
            return False
    def get_nearest_empty_parking_slot(self):
```

```
        """
        This method returns the minimum value from the
available_parking_slots min heap.
        :return: available_parking_slot:int The nearest
available parking slot to the entry terminal.
        """
        available_parking_slot =
heappop(self.available_parking_slots)
        return available_parking_slot
    def allocate_parking_slot(self, car):
        """
        This method is used to allocate the nearest
available parking slot to the car entering the entry
terminal.
        :param car:obj Object of the Car Class, to which
the parking slot has to be allocated.
        :return: parking_slot:int It returns the assigned
parking_slot to car or -1 if no parking_slot is
available.
        """
        # Checking if there are vacant slots available in
the parking lot
        if len(self.occupied_parking_slots) <
self.capacity:
            parking_slot =
self.get_nearest_empty_parking_slot()
            vehicle_registration_number =
car.get_registration_number()
            # A parking ticket is created for the car
with vehicle registration number, driver's age and
parking slot
            # assigned mentioned on the ticket.
            parking_ticket = ParkingTicket(car,
parking_slot)
            # Set the parking_ticket as value to key
vehicle_registration_number in the occupied_parking_slots
            # dictionary.

self.occupied_parking_slots[vehicle_registration_number]
= parking_ticket
        else:
            parking_slot = -1
        return parking_slot
    def deallocate_parking_slot(self,
parking_slot_number):
        """
```

```
        This method is used to deallocate the parking
slot number assigned to a car when it's vacating the
parking slot,
        and leaving from the exit terminal.
        :param parking_slot_number:int Parking Slot
Number which is getting vacated.
        :return: parking_ticket:obj or False:boolean
based on the deallocate status of the parking slot.
        """
        if len(self.occupied_parking_slots) > 0:
            key = -1
            # Looking up the vehicle_registration_number
and parking_ticket for the car which is allocated
            # the parking_slot_number passed as argument,
in occupied_parking_slots dictionary.
            for vehicle_registration_number,
parking_ticket in self.occupied_parking_slots.items():
                if parking_ticket.get_parking_slot() ==
parking_slot_number:
                    key = vehicle_registration_number
                    value = parking_ticket
                    break
            if key == -1:
                # If no car found which is allocated the
parking_slot_number passed as argument, return False.
                return False
            else:
                # Deleting the entry for the car vacating
the parking_slot and pushing the vacated parking slot
number
                # back to available_parking_slots min
heap.
                del self.occupied_parking_slots[key]
                heappush(self.available_parking_slots,
parking_slot_number)
                return value
        else:
            return False
    def issue_parking_ticket(self,
vehicle_registration_number, driver_age):
        """
        This method is used to issue parking ticket to
the car entering the parking lot from the entry terminal.
        :param vehicle_registration_number:str Vehicle
registration Number of the car
        :param driver_age:int Age of the driver driving
```

```
the car
        :return: parking_slot:int It returns the parking
slot number allocated to the car
        """
        # Create an object of Class Car by passing
vehicle_registration_number and driver_age as arguments.
        car = Car(vehicle_registration_number,
driver_age)
        parking_slot = self.allocate_parking_slot(car)
        return parking_slot
    def return_parking_ticket(self, parking_slot_number):
        """
        This method is used to deallocate the parking
slot number allocated to a car on returning parking
ticket,
        while vacating the parking slot and exiting
parking lot from the exit terminal.
        :param parking_slot_number:int Parking Slot
number that was allocated to the car.
        :return: parking_slot_number:int Vacated parking
slot number
        """
        return
self.deallocate_parking_slot(parking_slot_number)
    def
get_parking_slot_number_from_vehicle_registration_number(
self, vehicle_registration_number):
        """
        This method is used to look up the
occupied_parking_slots dictionary and return the assigned
parking_slot number
        from the parking ticket for the
vehicle_registration_number passed as argument.
        :param vehicle_registration_number:str Vehicle
Registration Number of the Car whose allocated
parking_slot is to
        be looked upon and returned
        :return: parking_slot:int Parking Slot number
occupied by the car with the vehicle_registration_number
passes
        as argument.
        """
        parking_slot = -1
        if vehicle_registration_number in
self.occupied_parking_slots:
            parking_slot =
```

```
self.occupied_parking_slots[vehicle_registration_number].
get_parking_slot()
        return parking_slot
    def
get_vehicle_registration_numbers_from_driver_age(self,
driver_age):
        """
        This method is used to look up the
occupied_parking_slots dictionary and return the vehicle
registration numbers
        for the cars whose driver's age matches with the
driver_age passed as argument in the parking ticket.
        :param driver_age:int Age of the driver
        :return: vehicle_registration_numbers:list List
of all vehicle_registration_numbers for which driver's
age
        matches with the driver_age passed as argument in
the parking ticket.
        """
        vehicle_registration_numbers = []
        for vehicle_registration_number, parking_ticket
in self.occupied_parking_slots.items():
            if parking_ticket.get_driver_age() ==
driver_age:

vehicle_registration_numbers.append(vehicle_registration_
number)
        return vehicle_registration_numbers
    def get_parking_slots_from_driver_age(self,
driver_age):
        """
        This method is used to look up the
occupied_parking_slots dictionary and return the
allocated parking_slots
        for the cars whose driver's age matches with the
driver_age passed as argument in the parking ticket.
        :param driver_age:int Age of the driver
        :return: parking_slots:list List of parking_slots
allocated to the cars for which driver's age
        matches with the driver_age passed as argument in
the parking ticket.
        """
        parking_slots = []
        for parking_ticket in
self.occupied_parking_slots.values():
            if parking_ticket.get_driver_age() ==
```

```
driver_age:

parking_slots.append(parking_ticket.get_parking_slot())
        return parking_slots
    def parse_commands(self, query):
        """
        This method takes query as input and executes the
query on the predefined set of commands and matching
methods
        to be executed, and prints the output to the
console or writes it to a output file based on the mode
selected.
        if query.startswith('Create_parking_lot'):
            try:
                capacity = int(query.split(' ')[1])
                status =
self.create_parking_slots(capacity)
                if status:
                    print(f'Created parking of {capacity}
slots')
            except Exception as exception:
                print(f'Error in Query - {query} :
{exception}')
        elif query.startswith('Park'):
            try:
                vehicle_registration_number =
query.split(' ')[1]
                driver_age = query.split(' ')[3]
                result =
self.issue_parking_ticket(vehicle_registration_number,
driver_age)
                if result == -1:
                    print('Sorry, Parking Lot is full, No
Parking Slots Available.')
                else:
                    print(
                        f'Car with vehicle registration
number "{vehicle_registration_number}" has been parked at
'
                        f'slot number {result}')
            except Exception as exception:
                print(f'Error in Query - {query} :
{exception}')
        elif query.startswith('Leave'):
            try:
                leaving_parking_slot = int(query.split('
```

```
')[1])
                parking_ticket =
self.return_parking_ticket(leaving_parking_slot)
                if parking_ticket:
                    print(f'Slot number
{parking_ticket.get_parking_slot()} vacated, the car with
vehicle registration '
                            f'number
"{parking_ticket.get_vehicle_registration_number()}" left
the space, the driver of '
                            f'the car was of age
{parking_ticket.get_driver_age()}')
                else:
                    print(f'Slot number {leaving_parking_slot}
cannot be vacated.')
            except Exception as exception:
                print(f'Error in Query - {query} :
{exception}')
        elif
query.startswith('Slot_number_for_car_with_number'):
            try:
                car_registration_number = query.split(' ')[1]
                parking_slot =
self.get_parking_slot_number_from_vehicle_registration_number(
car_registration_number)
                if parking_slot == -1:
                    print('No parked car matches the query')
                else:
                    print(parking_slot)
            except Exception as exception:
                print(f'Error in Query - {query} :
{exception}')
        elif
query.startswith('Vehicle_registration_number_for_driver_
of_age'):
            try:
                driver_age = int(query.split(' ')[1])
                vehicle_registration_numbers =
self.get_vehicle_registration_numbers_from_driver_age(dri
ver_age)
                if len(vehicle_registration_numbers) > 0:

print(','.join(vehicle_registration_numbers))
                else:
                    print('No parked car matches the
query')
```

```
            except Exception as exception:
                print(f'Error in Query - {query} :
{exception}')
        elif
query.startswith('Slot_numbers_for_driver_of_age'):
            try:
                driver_age = int(query.split(' ')[1])
                parking_slots =
self.get_parking_slots_from_driver_age(driver_age)
                if len(parking_slots) > 0:
                    print(','.join([str(parking_slot) for
parking_slot in parking_slots]))
                else:
                    print('No parked car matches the
query')
            except Exception as exception:
                print(f'Error in Query - {query} :
{exception}')
        else:
            print('Query not recognized.')
if __name__ == '__main__':
    # Creates an object of Main Class Parking Management
    parking_management = ParkingManagement()
    parser = argparse.ArgumentParser()
    parser.add_argument('--input_file', action="store",
required=True, dest='input_file', help="Input File")
    parser.add_argument('--output_file', action="store",
required=False, dest='output_file', help="Output File")
    args = parser.parse_args()
    # if output_file is argument is specified, all the
console output will be written to the output_file.
    if args.output_file:
        sys.stdout = open(args.output_file, "w")
```

7. SOFTWARE ENVIRONMENT

What is Python :-

Below are some facts about Python.

Python is currently the most widely used multi-purpose, high-level programming language.

Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.

Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc.

The biggest strength of Python is huge collection of standard library which can be used .

- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

Advantages of Python :-

Let’s see how Python dominates over other languages.

1. Extensive Libraries

Python downloads with an extensive library. So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be extended to other languages. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add scripting capabilities to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers more productive than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world

6. Simple and Easy

When working with Java, you may have to create a class to print 'Hello World'. But in Python, just a print statement will do. It is also quite easy to learn**,** understand**,** and code**.** This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and indentation is mandatory**.** This further aids the readability of the code.

8. Object-Oriented

This language supports both the procedural and object-oriented programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

9. Free and Open-Source

Like we said earlier, Python is freely available**.** But not only can you download Python for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to code only once, and you can run it anywhere. This is called Write Once Run Anywhere (WORA). However, you need to be careful enough not to include any system-dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, debugging is easier than in compiled languages.

Python Development Steps : -

Guido Van Rossum published the first version of Python code (version 0.9.0) at alt.sources in February 1991. This release included already exception handling, functions, and the core data types of list, dict, str and others. It was also object oriented. Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked.Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehensions, a full garbage collector and it was supporting unicode.Python flourished for another 8 years in the versions 2.x before the next major release as Python 3.0 (also known as "Python 3000" and "Py3K") was released. Python 3 is not backwards compatible with Python 2.x. The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to fulfilling the 13th law of the Zen of Python: "There should be one -- and preferably only one -- obvious way to do it."Some changes in Python 7.3:

- Print is now a function
- Views and iterators instead of lists
- The rules for ordering comparisons have been simplified. E.g. a heterogeneous list.
- There is only one integer type left, i.e. int. long is int as well.
- The division of two integers returns a float instead of an integer.
- Text Vs. Data Instead Of Unicode Vs. 8-bit

Purpose :-

We demonstrated that our approach enables successful segmentation of intra-retinal layers—even with low-quality images containing speckle noise, low

contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.

Python is Interactive − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Modules Used in Project :-

Tensorflow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and is also used for machine learning applications such as neural networks. It is used for both research and production at Google.

TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

Numpy

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution

towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object oriented interface or via a set of functions familiar to MATLAB users.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in

1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.

Python is Interactive − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Install Python Step-by-Step in Windows and Mac :

Python a versatile programming language doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: https://www.python.org

Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.

Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.

To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.

Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.7 to PATH.

Step 3: Click on Install NOW After the installation is successful. Click on Close.

With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.

Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type python – V and press Enter.

Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.

Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. Click on File > Click on Save

Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. enter print**.**

8. SYSTEM TESTING

8.1 TESTING STRATEGIES

UNIT TESTING

Unit testing, a testing technique using which individual modules are tested to determine if there are issues by the developer himself.. it is concerned with functional correctness of the standalone modules. The main aim is to isolate each unit of the system to identify, analyze and fix the defects.

Unit Testing Techniques:

Black Box Testing - Using which the user interface, input and output are tested.

White Box Testing –Used to test each one of those functions behavior is tested.

DATA FLOW TESTING

Data flow testing is a family of testing strategies based on selecting paths through the program's control flow in order to explore sequence of events related to the status of Variables or data object. Dataflow Testing focuses on the points at which variables receive and the points at which these values are used.

INTEGRATION TESTING

Integration Testing done upon completion of unit testing, the units or modules are to be integrated which gives raise too integration testing. The purpose of integration testing is to verify the functional, performance, and reliability between the modules that are integrated.

BIG BANG INTEGRATION TESTING

Big Bang Integration Testing is an integration testing Strategy wherein all units are linked at once, resulting in a complete system. When this type of testing strategy is adopted, it is difficult to isolate any errors found, because attention is not paid to verifying the interfaces across individual units.

USER INTERFACE TESTING

User interface testing, a testing technique used to identify the presence of defects is a product/software under test by Graphical User interface [GUI].

8.2 TEST CASES:

S.NO	INPUT	If available	If not available
1	User signup	User get registered into the application	There is no process
2	User signin	User get login into the application	There is no process
3	Detection	Vehicle detection completed	There is no process
4	Parking	Vehicles parking places arranged	There is no process

Table 8.2.1 Test Cases

9. SCREENSHOTS

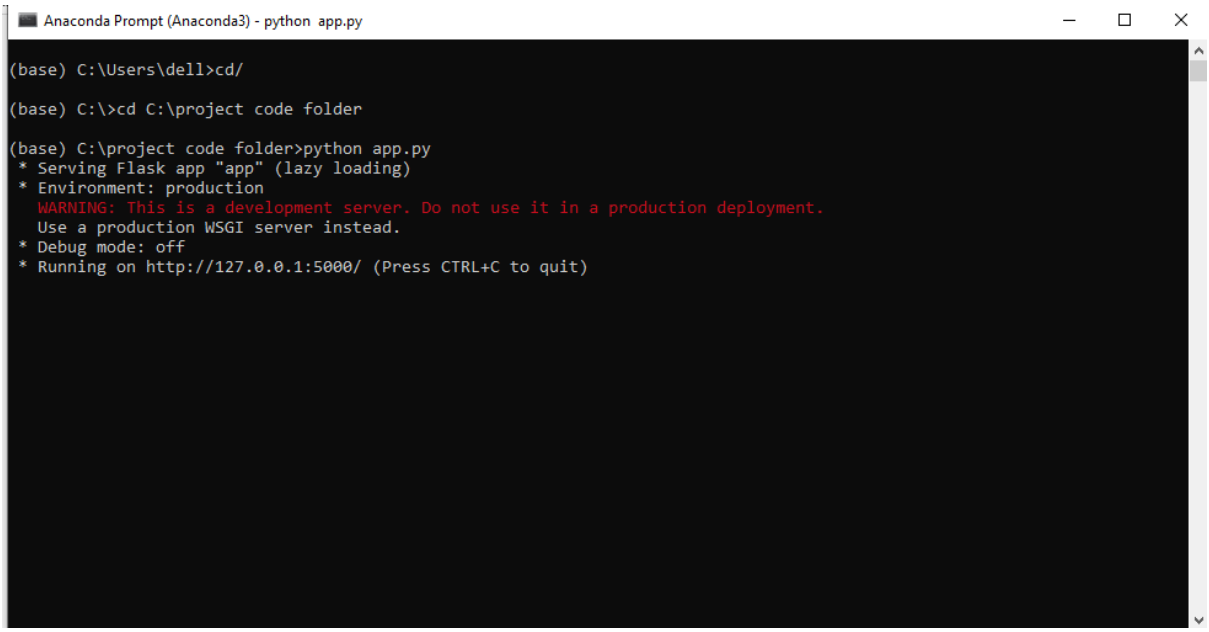

Fig 9.1 command prompt

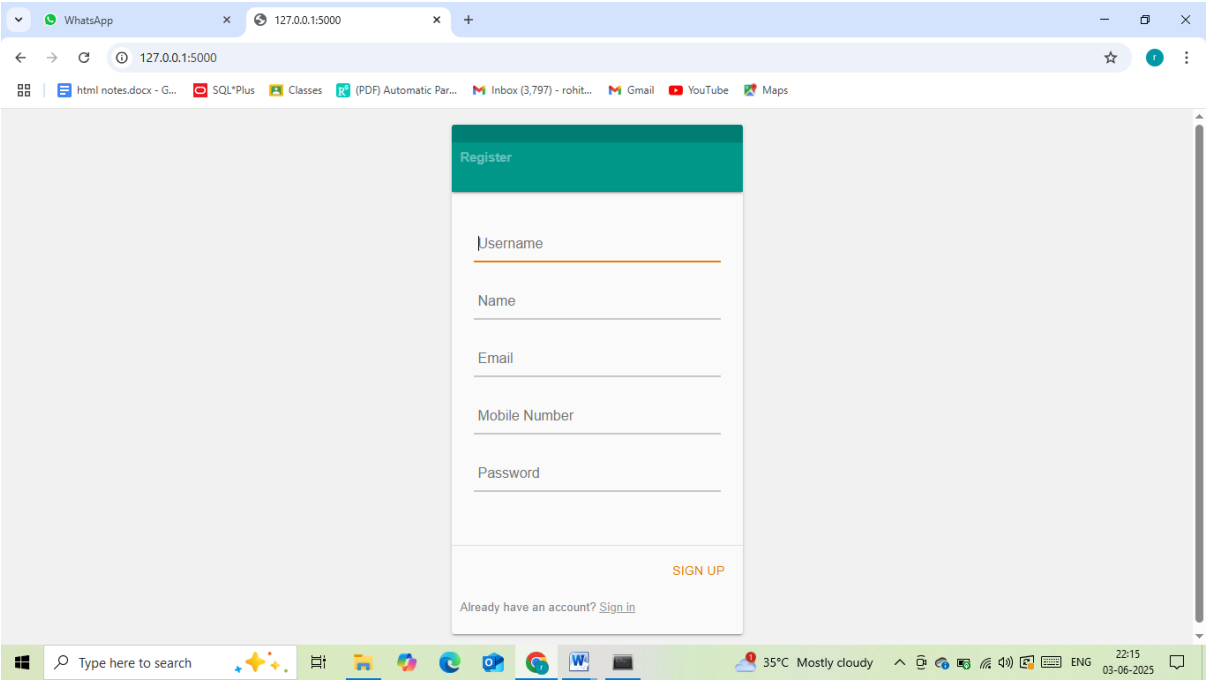

Fig 9.2 Sign up page

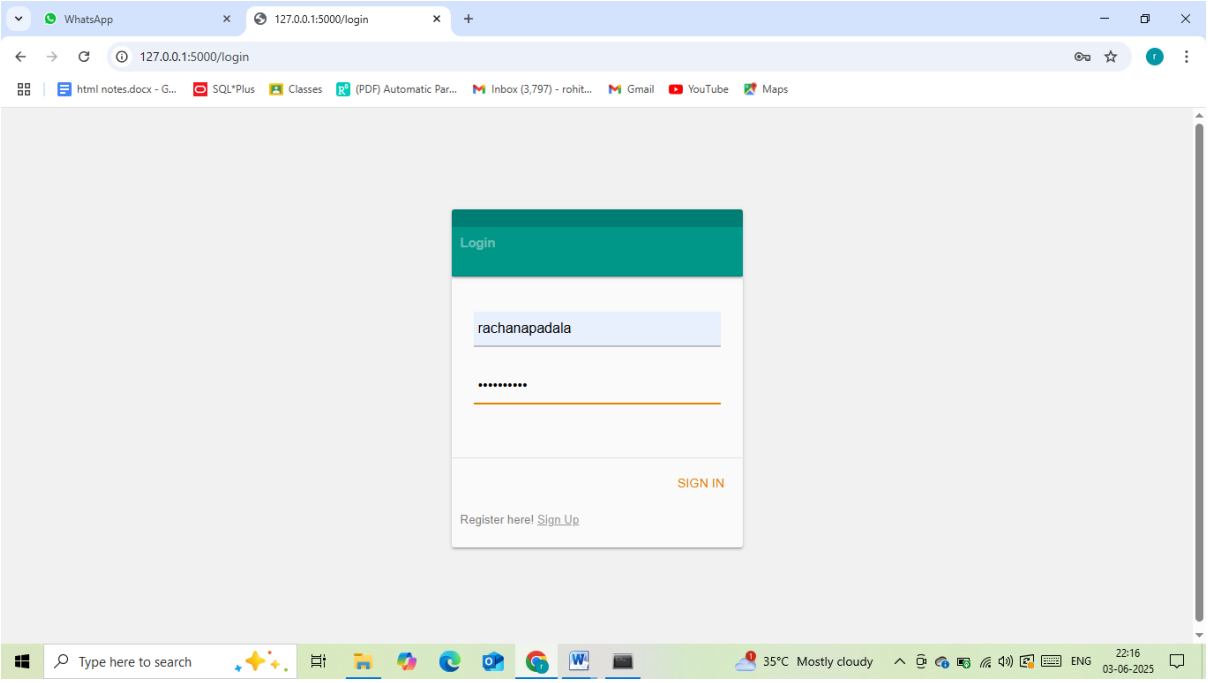

Fig 9.3 sign in page

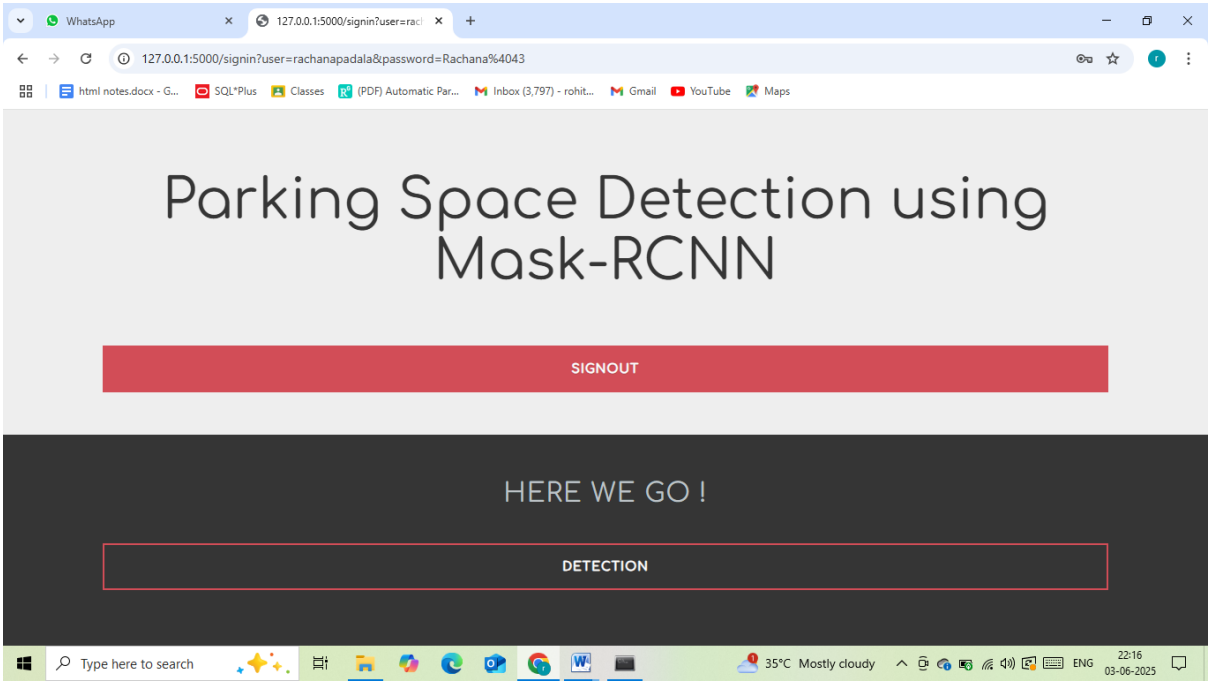

Fig 9.4 Home page after sign in

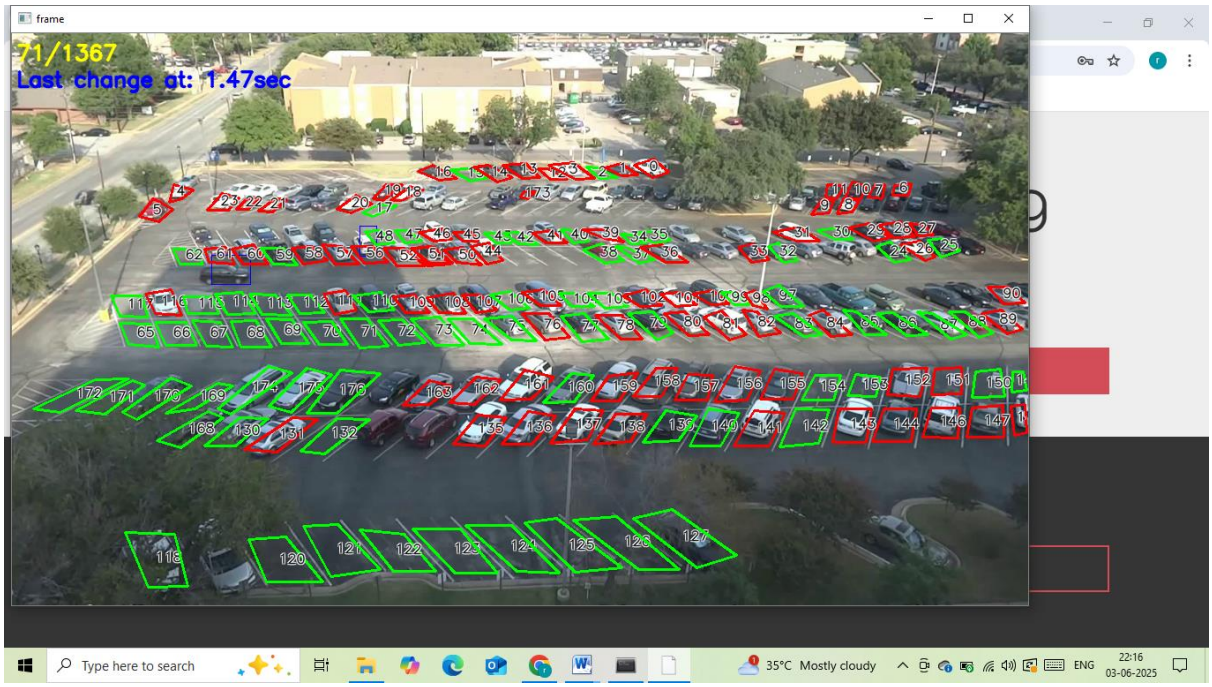

Fig 9.5 Output Detection

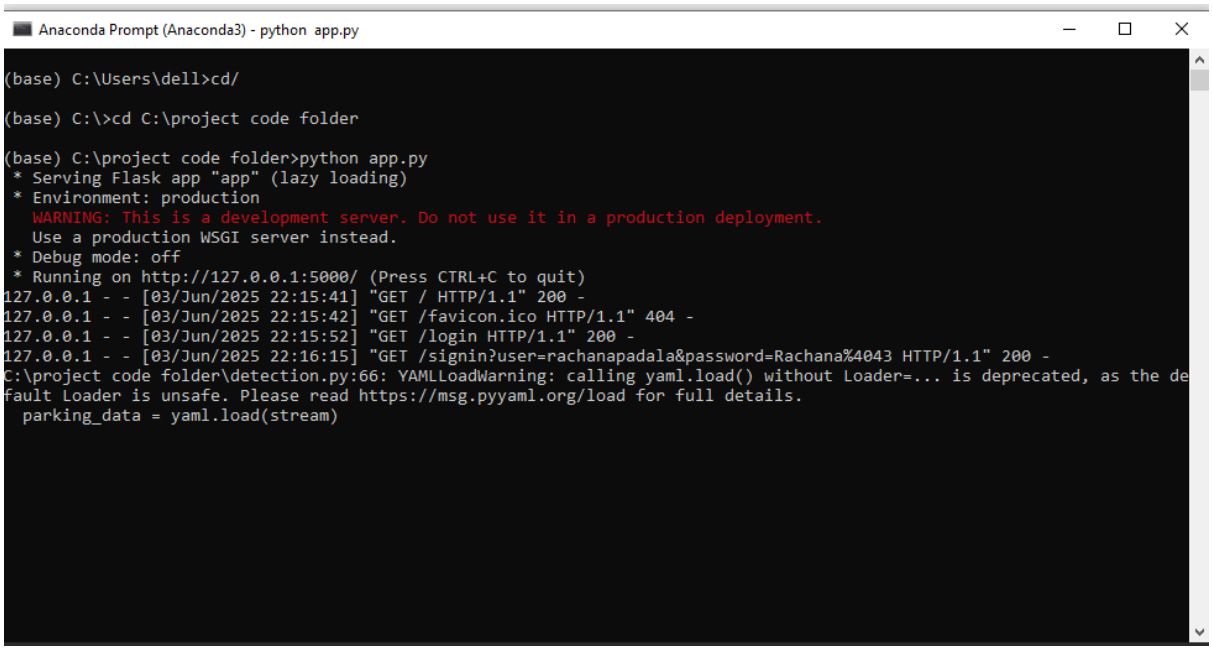

Fig 9.6 Anaconda prompt

10. CONCLUSION

This paper presents a multicamera system for the management of vacant parking places by means of vehicle detection and their corresponding mapping into the parking spaces of a parking lot. The system has been designed so that existing parking lot security cameras can be used for the proposed system after a simple configuration, without the need for a complete new camera deployment. The designed system faces more complicated scenarios than the ones tackled in the state of the art: almost total occlusions and climatic changes (cloudy scenarios, rain, snow...), that limits/reduces their performance. In this scenario with such a variable background it is not possible to carry out a precise background extraction, nor it is possible to label and define the region of each place as some parked vehicles completely occlude some of the spots behind them. In addition, the consideration of a multicamera scenario, which, as far as we know, has not been reported before for this type of systems, is added. A new dataset has been recorded and synchronized. The publicly available dataset is composed by the generated parking vehicle models, the recorded frames and the ground truth files. There are multiple future work lines to improve the proposed system. With respect to the combination, we have chosen a simple technique using normalized sigmoid functions, therefore different functions could be studied in order to optimize the combination or fusion of the different information sources. Also a new dataset with more cameras and with different spatial configurations could be recorded to see the behavior of the system in those situations.

11.REFERENCES

[1] T. Fabian, “An algorithm for parking lot occupation detection,” in Proc. Comput. Inf. Syst. Ind. Manage. Appl., Jun. 2018, pp. 165–170.

[2] C.-C. Huang and S.-J. Wang, “A hierarchical Bayesian generation framework for vacant parking space detection,” IEEE Trans. Circuits Syst. Video Technol., vol. 20, no. 12, pp. 1770–1785, Dec. 2021.

[3] X. Wang and A. R. Hanson, “Parking lot analysis and visualization from aerial images,” in Proc. 4th IEEE Workshop Appl. Comput. Vis. (WACV), Oct. 2020, pp. 36–41.

[4] K. Yamada and M. Mizuno, “A vehicle parking detection method using image segmentation,” Electron. Commun. Jpn., vol. 84, no. 10, pp. 25–34, Oct. 2019.

[5] C.-H. Lee, M.-G. Wen, C.-C. Han, and D.-C. Kou, “An automatic monitoring approach for unsupervised parking lots in outdoors,” in Proc. 39th Annu. Int. Carnahan Conf. Secur. Technol. (CCST), Oct. 2005, pp. 271–274.

[6] D. B. Bong, K. C. Ting, and N. Rajaee, “Car-park occupancy information system,” Real-Time Technol. Appl. Symp., Apr. 2016, pp. 1–4.

[7] B. L. Bong, K. C. Ting, and K. C. Lai, “Integrated approach in the design of car park occupancy information system (coins),” Int. J. Comput. Sci., vol. 35, no. 1, pp. 1–8, Feb. 2018.

[8] S. F. Lin, Y. Y. Chen, and S. C. Liu, “A vision-based parking lot management system,” in Proc. Int. Conf. Syst., Man Cybern., vol. 4, Oct. 2016, pp. 2897–2902.

[9] L. C. Chen, J. W. Hsieh, W. R. Lai, C. X. Wu, and S. Y. Chen, “Vision-based vehicle surveillance and parking lot management using multiple cameras,” in Proc. Int. Conf. Intell. Inf. Hiding Multimedia Signal Process., Oct. 2021, pp. 631–634.

[10] K. Blumer, H. R. Halaseh, M. U. Ahsan, H. Dong, and N. Mavridis, Cost-Effective Single-Camera Multi-Car Parking Monitoring and Vacancy Detection towards Real-World Parking Statistics and Real-Time Reporting. Berlin, Germany: Springer, 2022, pp. 506–515.

[11] Awan, Faraz Malik, et al. "A comparative analysis of machine/deep learning models for parking space availability prediction." *Sensors* 20.1 (2020): 322.

[12] Huang, Chen, et al. "Visual detection and image processing of parking space based on deep learning." *Sensors* 22.17 (2022): 6672.